FINAL INTERNSHIP PROJECT REPORT

Intelligent Invoice / Quotation Understanding Engine

AI/ML Internship Task

Neirah Tech Solution

Prepared By : Ajanthan T.
Institution : University of Jaffna
Student ID : 2021/E/192

Repository : Intelligent-Invoice/Quotation-Engine
Submission Date : 28 September 2026

1 Executive Summary

This project implements an Intelligent Invoice / Quotation Understanding Engine using Python and FastAPI. The system accepts PDF and image documents, preprocesses them for OCR, extracts text using PaddleOCR, classifies each document as INVOICE, QUOTATION or UNKNOWN, extracts structured fields and line items, validates financial consistency, calculates confidence information and returns a structured JSON response. The design emphasizes safe uncertainty handling like unsupported, corrupt, low-confidence, incomplete or financially inconsistent documents are surfaced for manual review rather than silently accepted.

The completed prototype was tested against standard invoices, alternative quotation layouts, rotated and low-resolution inputs, deliberately incorrect totals, missing required fields, unrelated documents and API-level error scenarios. The application is exposed through FastAPI with Swagger/OpenAPI documentation and has also been verified inside a Docker container.

2 Project Objective

The objective was to develop a maintainable document-understanding service that can transform invoices and quotations into structured, machine-readable data without relying on hard-coded document coordinates or filename-based classification. The engine was designed to be suitable as a realistic prototype that could later be integrated with accounting, ERP or document-processing workflows.

- Accept PDF, JPG, JPEG and PNG documents.
- Preprocess scanned, rotated and low-quality documents.
- Perform OCR and preserve confidence information.
- Classify documents as INVOICE, QUOTATION or UNKNOWN.
- Extract key business fields and line items from multiple layouts.
- Validate required fields and financial calculations.
- Return PROCESSED, MANUAL_REVIEW_REQUIRED or INVALID status.
- Expose the complete flow through a FastAPI endpoint and API documentation.

3 Scope

3.1 In Scope

- Invoice and quotation understanding from PDF/image inputs.
- Preprocessing, OCR, classification, field extraction, line-item extraction, validation, confidence scoring and structured output.
- Controlled error handling for unsupported and corrupt files.
- FastAPI API integration and Swagger documentation.
- Automated tests and robustness scenarios.
- Docker packaging for reproducible deployment.

3.2 Out of Scope

- ERP implementation, payment processing or payment gateways.
- Complex user management, messaging or workflow orchestration.
- Large frontend/mobile applications.
- Training a large custom OCR or language model from scratch.

4 Technology Stack

Technology	Purpose
Python 3.11	Primary implementation language
FastAPI	REST API and Swagger/OpenAPI documentation
PaddleOCR	OCR/text detection and recognition
PaddlePaddle 3.2.2	Runtime used by PaddleOCR
OpenCV	Image preprocessing and test-fixture generation
PyMuPDF	PDF rendering and page conversion
Pydantic	API response and validation models
Pytest	Automated unit and API testing
Uvicorn	ASGI application server
Docker	Containerized deployment and reproducibility

Table 1: Technology stack used

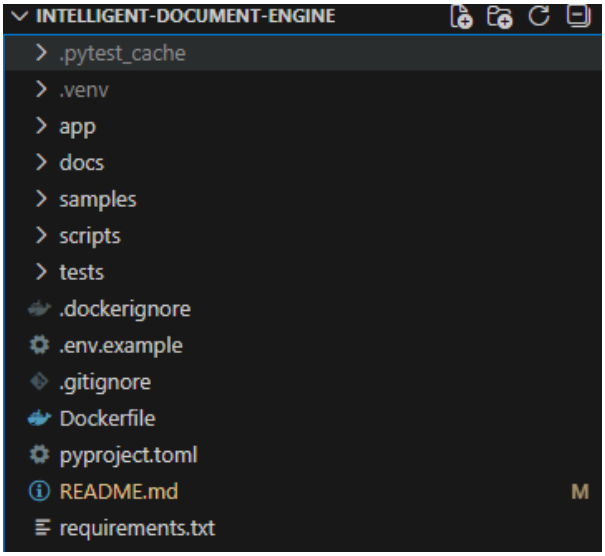


Figure 1: Project structure in Visual Studio Code

5 System Architecture

The application follows a modular pipeline. Each stage has a focused responsibility, which makes the system easier to test, maintain and extend. The high-level data flow is shown below.

Stage	Responsibility
1. Upload / Input Validation	Accept supported files, reject unsupported or corrupt inputs safely.
2. Preprocessing	Normalize PDF/image inputs, convert pages, enhance readability and handle rotation/quality issues.
3. OCR	Extract text, bounding boxes, page information and OCR confidence.
4. Classification	Use content and structural evidence to classify INVOICE, QUOTATION or UNKNOWN.
5. Field Extraction	Extract document metadata, parties, totals, currency and other structured fields.
6. Line Item Extraction	Use spatial row/column reasoning to extract description, quantity, unit price and line total where available.
7. Validation	Check missing required fields, arithmetic consistency, confidence and document quality.
8. Status Resolution	Return PROCESSED, MANUAL_REVIEW_REQUIRED or INVALID.
9. API Response	Serialize the complete result as structured JSON.

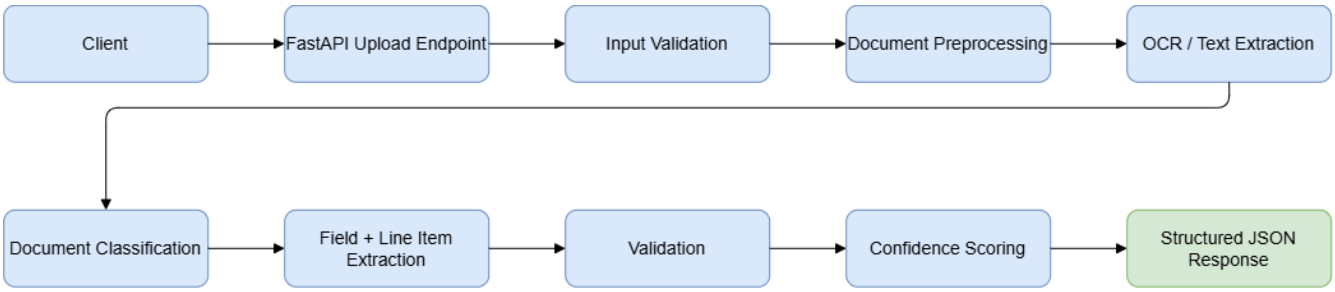


Figure 2: System architecture / processing pipeline

6 Implementation Details

6.1 Input Validation and Processing

The upload layer validates supported extensions and handles invalid input using controlled exceptions. Preprocessing supports both PDF and image documents. PDF pages are rasterized for OCR, while images are converted and normalized before recognition. Grayscale preprocessing is used internally, while the OCR provider converts images back to a compatible BGR representation when necessary.

- Supported inputs: PDF, JPG, JPEG and PNG.
- Multi-page PDF processing is supported.
- Corrupt and unsupported uploads are rejected cleanly.
- Rotation, low resolution and blur were included in robustness testing.

6.2 OCR/Text Extraction

PaddleOCR is used as the OCR provider. OCR output is normalized into application-specific models containing recognized text, confidence, page number and bounding-box geometry. This spatial information is important for locating values relative to labels and for reconstructing line-item tables.

During development, PaddlePaddle 3.3.x produced a oneDNN/PIR runtime error in the tested environment. The working runtime was pinned to PaddlePaddle 3.2.2. The container and local setup therefore document the known-good dependency combination rather than relying on an untested latest version.

6.3 Document Classification

Classification is evidence-based. It combines invoice- or quotation-specific signals with common business structure signals instead of using filenames or a single keyword. When the evidence is not strong enough, the document remains UNKNOWN. This avoids forcing unrelated documents into a supported business category.

6.4 Field Extraction

The field extractor uses text patterns, semantic label matching, and spatial relationships such as values located to the right of or below a label. It supports document number, date, vendor/company, customer, address, currency, subtotal, discount, tax, additional charges and grand total. The implementation avoids fixed coordinates so that different document layouts can be handled by the same extraction logic.

6.5 Line Item Extraction

Line items are reconstructed from OCR geometry by grouping text into rows and identifying header columns. The extractor supports flexible tables where quantity and unit price may be absent, as long as description and line-total information can be identified. This was necessary for the second quotation layout, which used DESCRIPTION / TAXED / AMOUNT rather than a traditional quantity/unit-price table.

6.6 Validation and Confidence

Validation checks required fields, arithmetic consistency, OCR confidence, classification confidence and financial relationships. Financial checks use a small decimal tolerance. Missing or inconsistent information does not get silently repaired; the issue is returned in validation_issues and the document can be routed to manual review.

Validation Example	Expected Behaviour
Missing document number/date/total	Generate a missing-field issue and require manual review.
Incorrect line total	Compare quantity x unit price with extracted line total.
Incorrect subtotal	Compare subtotal with available line totals where applicable.
Incorrect grand total	Compare subtotal - discount + tax + charges with grand total.
Low OCR/classification confidence	Generate confidence-related review issue.
Unrelated document	Keep classification as UNKNOWN and require review rather than forcing a type.

Table 2: Validation example with expected behaviour

6.7 FastAPI Integration

The processing stages are orchestrated through a document pipeline and exposed using FastAPI. The main analysis endpoint is POST /api/documents/analyze. Pydantic models serialize classification, field values, field confidence, line items, validation issues and final status. The OCR service is cached so that the heavy model does not need to be re-created for every API request.

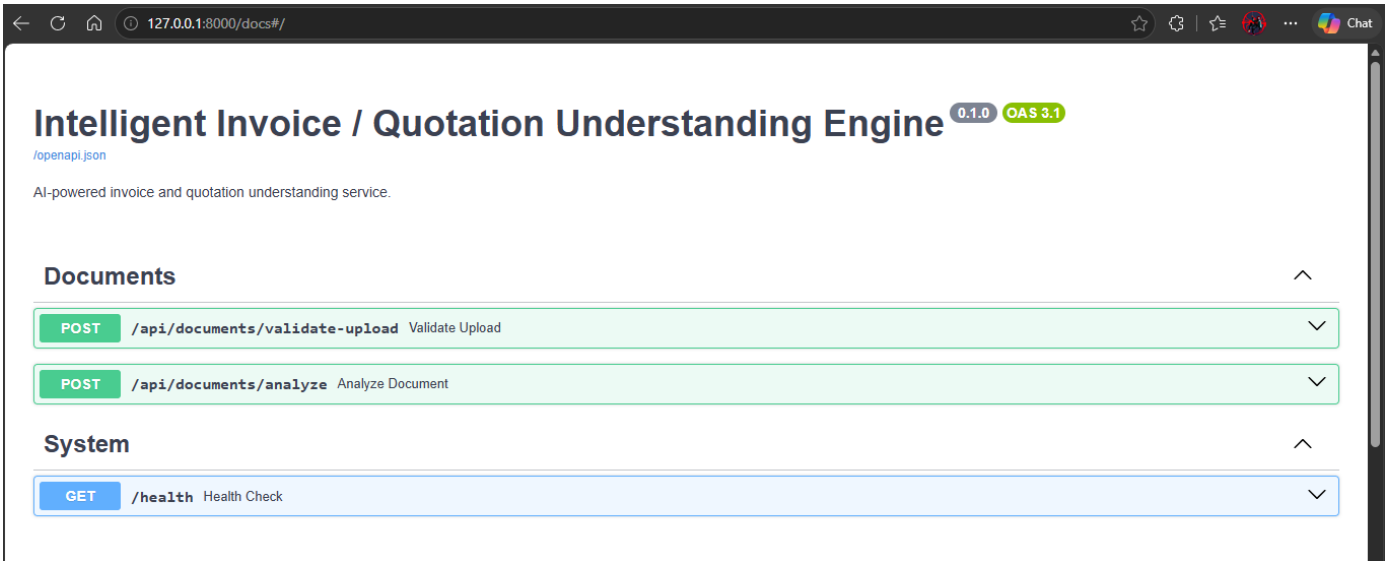


Figure 3: Swagger / OpenAPI interface for the FastAPI service

7 Testing Strategy

Testing was performed incrementally throughout development. Unit tests cover preprocessing, OCR integration abstractions, classification, field extraction, line-item extraction, validation and API behavior. Regression tests were added when generalization improvements affected an earlier working invoice.

At the end of the robustness checkpoint, the complete automated test suite passed 57 tests.

```
tests/unit/test_ocr_service.py::test_multi_page_ocr PASSED                                    [ 69%]
tests/unit/test_ocr_service.py::test_empty_ocr_result PASSED                                  [ 71%]
tests/unit/test_ocr_service.py::test_prepare_grayscale_image_for_paddle PASSED                [ 72%]
tests/unit/test_ocr_service.py::test_prepare_bgr_image_keeps_shape PASSED                     [ 74%]
tests/unit/test_party_extraction.py::test_bill_to_block PASSED                                [ 76%]
tests/unit/test_party_extraction.py::test_customer_block PASSED                               [ 77%]
tests/unit/test_party_extraction.py::test_sold_to_block PASSED                                [ 79%]
tests/unit/test_preprocessing.py::test_decode_valid_png PASSED                                [ 81%]
tests/unit/test_preprocessing.py::test_decode_corrupt_image PASSED                            [ 83%]
tests/unit/test_preprocessing.py::test_preprocess_returns_grayscale PASSED                    [ 84%]
tests/unit/test_preprocessing.py::test_rotate_90_degrees PASSED                               [ 86%]
tests/unit/test_preprocessing.py::test_render_valid_pdf PASSED                                [ 88%]
tests/unit/test_preprocessing.py::test_corrupt_pdf_rejected PASSED                            [ 89%]
tests/unit/test_validation.py::test_valid_document_is_processed PASSED                        [ 91%]
tests/unit/test_validation.py::test_missing_document_number_requires_review PASSED            [ 93%]
tests/unit/test_validation.py::test_line_total_mismatch_detected PASSED                       [ 94%]
tests/unit/test_validation.py::test_subtotal_mismatch_detected PASSED                         [ 96%]
tests/unit/test_validation.py::test_grand_total_mismatch_detected PASSED                      [ 98%]
tests/unit/test_validation.py::test_low_ocr_confidence_requires_review PASSED                 [100%]

============================================================ 59 passed in 8.40s ============================================================
```

Figure 4: Automated test suite result

8 Final Demonstration Scenario Results

The final report serves as the demonstration evidence. The following scenarios correspond to the required prototype behaviours and show how the engine reacts to normal, degraded, inconsistent, incomplete and unsupported documents. Replace each placeholder with a screenshot from the running API and retain the caption.

8.1 Scenario 1 – Valid Invoice

A clean invoice was processed through the complete pipeline. The system classified it as INVOICE, extracted document number US-001, date 11/02/2019, vendor East Repair inc., customer John Smith, subtotal 145.00, tax 9.06, grand total 154.06 and three line items. After the final regression fix, the document returned PROCESSED with no validation issues.



INVOICE

East Repair Inc.
1912 Harvest Lane
New York, NY 12210

BILL TO
John Smith
2 Court Square
New York, NY 12210

SHIP TO
John Smith
3787 Pineview Drive
Cambridge, MA 12210

INVOICE #	US-001
INVOICE DATE	11/02/2019
P.O.#	2312/2019
DUE DATE	26/02/2019

QTY	DESCRIPTION	UNIT PRICE	AMOUNT
1	Front and rear brake cables	100.00	100.00
2	New set of pedal arms	15.00	30.00
3	Labor 3hrs	5.00	15.00

Subtotal	145.00
Sales Tax 6.25%	9.06
TOTAL	$154.06

TERMS & CONDITIONS
Payment is due within 15 days

Please make checks payable to: East Repair Inc.

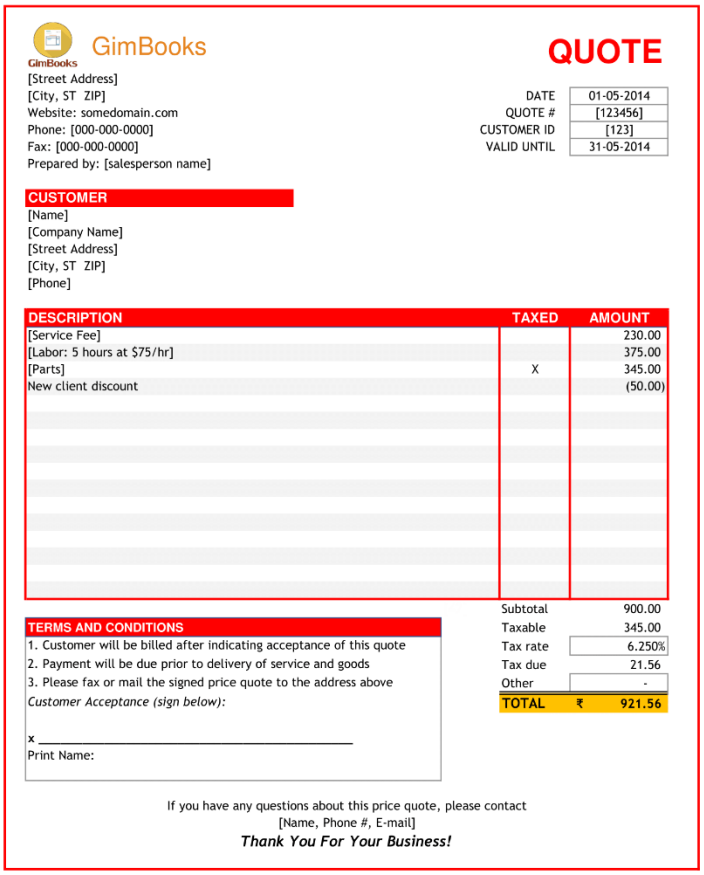
Figure 5: Valid invoice sample used for end-to-end analysis

```
{
  "document_type": "INVOICE",
  "confidence": 0.883,
  "fields": {
    "document_number": {
      "value": "US-001",
      "confidence": 0.893
    },
    "document_date": {
      "value": "11/02/2019",
      "confidence": 0.638
    },
    "vendor_name": {
      "value": "East Repair inc.",
      "confidence": 0.913
    },
    "currency": {
      "value": "$",
      "confidence": 0.9
    },
    "subtotal": {
      "value": 145,
      "confidence": 0.95
    },
    "discount": {
      "value": null,
      "confidence": 0
    },
    "tax": {
      "value": 9.06,
      "confidence": 0.896
    },
    "additional_charges": {
      "value": null,
      "confidence": 0
    },
    "grand_total": {
      "value": 154.06,
      "confidence": 0.95
    },
    "customer_name": {
      "value": "John Smith",
      "confidence": 0.993
    },
    "address": {
      "value": "2 Court Square, New York, NY 12210",
      "confidence": 0.993
    }
  },
  "line_items": [
    {
      "description": "Front and rear brake cables",
      "quantity": 1,
      "unit_price": 100,
      "line_total": 100,
      "confidence": 0.981
    },
    {
      "description": "New set of pedal arms",
      "quantity": 2,
      "unit_price": 15,
      "line_total": 30,
      "confidence": 0.998
    },
    {
      "description": "Labor 3hrs",
      "quantity": 3,
      "unit_price": 5,
      "line_total": 15,
      "confidence": 0.999
    }
  ],
  "validation_issues": [],
  "status": "PROCESSED"
}
```


Figure 6: Structured JSON result for the valid invoice

8.2 Scenario 2 – Different Layout Quotation

A quotation with a substantially different structure was used to confirm that extraction did not depend on fixed invoice coordinates. The system classified the document as QUOTATION and extracted document number [123456], date 01-05-2014, vendor GimBooks, subtotal 900.00, tax 21.56, grand total 921.56, customer information and four line items. Quantity and unit price remained null where that layout did not provide traditional columns. Currency remained null because the OCR result did not reliably capture the rupee symbol; the system did not invent a currency from unrelated line text.



GimBooks

[Street Address]
[City, ST ZIP]
Website: somedomain.com
Phone: [000-000-0000]
Fax: [000-000-0000]
Prepared by: [salesperson name]

QUOTE

DATE	01-05-2014
QUOTE #	[123456]
CUSTOMER ID	[123]
VALID UNTIL	31-05-2014

CUSTOMER

[Name]
[Company Name]
[Street Address]
[City, ST ZIP]
[Phone]

DESCRIPTION	TAXED	AMOUNT
[Service Fee]		230.00
[Labor: 5 hours at $75/hr]		375.00
[Parts]	X	345.00
New client discount		(50.00)

Subtotal		900.00
Taxable		345.00
Tax rate		6.250%
Tax due		21.56
Other		-
TOTAL	₹	921.56

TERMS AND CONDITIONS

1. Customer will be billed after indicating acceptance of this quote
2. Payment will be due prior to delivery of service and goods
3. Please fax or mail the signed price quote to the address above

Customer Acceptance (sign below):

x _____
Print Name:

If you have any questions about this price quote, please contact
[Name, Phone #, E-mail]

Thank You For Your Business!

Figure 7: Different-layout quotation sample


```
{
  "document_type": "QUOTATION",
  "confidence": 0.873,
  "fields": {
    "document_number": {
      "value": "[123456]",
      "confidence": 0.946
    },
    "document_date": {
      "value": "01-05-2014",
      "confidence": 0.65
    },
    "vendor_name": {
      "value": "GimBooks",
      "confidence": 0.8
    },
    "currency": {
      "value": null,
      "confidence": 0
    },
    "subtotal": {
      "value": 900,
      "confidence": 0.95
    },
    "discount": {
      "value": null,
      "confidence": 0
    },
    "tax": {
      "value": 21.56,
      "confidence": 0.95
    },
    "additional_charges": {
      "value": null,
      "confidence": 0
    },
    "grand_total": {
      "value": 921.56,
      "confidence": 0.95
    },
    "customer_name": {
      "value": "[Name]",
      "confidence": 0.995
    },
    "address": {
      "value": "[Company Name], [Street Address], [City, ST ZIP]",
      "confidence": 0.995
    }
  },
  "line_items": [
    {
      "description": "[Service Fee]",
      "quantity": null,
      "unit_price": null,
      "line_total": 230,
      "confidence": 0.986
```

```
    },
    {
      "description": "[Labor: 5 hours at $75/hr]",
      "quantity": null,
      "unit_price": null,
      "line_total": 375,
      "confidence": 0.984
    },
    {
      "description": "[Parts]",
      "quantity": null,
      "unit_price": null,
      "line_total": 345,
      "confidence": 0.973
    },
    {
      "description": "New client discount",
      "quantity": null,
      "unit_price": null,
      "line_total": -50,
      "confidence": 1
    }
  ],
  "validation_issues": [],
  "status": "PROCESSED"
}
```


Figure 8: Structured JSON result for the different-layout quotation

8.3 Scenario 3 – Rotated/Low-Quality Document

Degraded variants were generated from a known clean invoice so the expected content was known. A 90-degree rotated version continued to process successfully, while low-resolution and blurred variants were routed to MANUAL_REVIEW_REQUIRED when confidence or extraction quality became insufficient. This demonstrates that the system surfaces uncertainty rather than treating degraded OCR output as fully reliable.



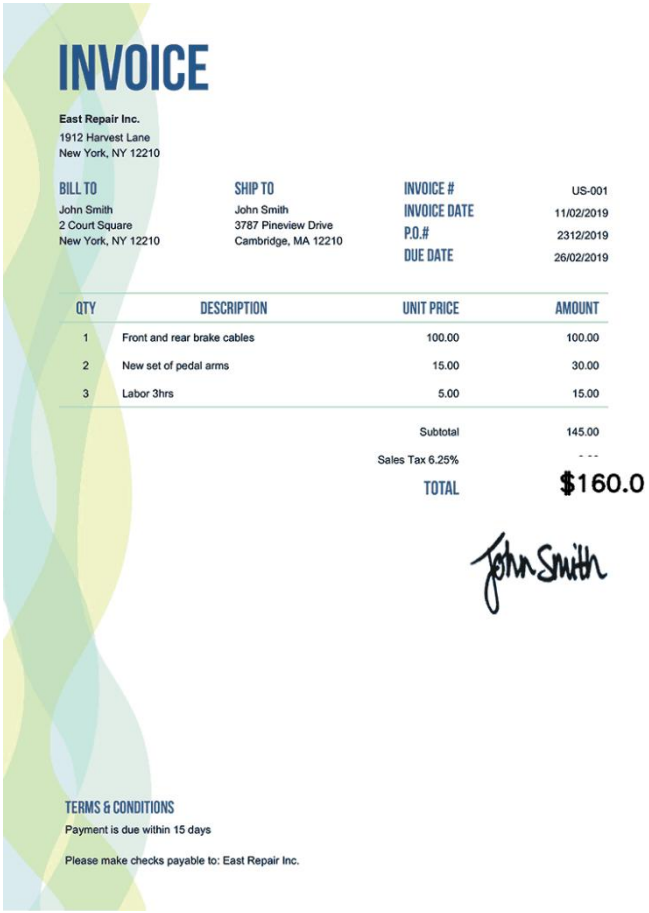
Figure 9: Low-resolution invoice test input

```
{
  "document_type": "UNKNOWN",
  "confidence": 0.6,
  "fields": {
    "document_number": {
      "value": null,
      "confidence": 0
    },
    "document_date": {
      "value": null,
      "confidence": 0
    },
    "vendor_name": {
      "value": "ICE",
      "confidence": 0.8
    },
    "currency": {
      "value": null,
      "confidence": 0
    },
    "subtotal": {
      "value": null,
      "confidence": 0
    },
    "discount": {
      "value": null,
      "confidence": 0
    },
    "tax": {
      "value": null,
      "confidence": 0
    },
    "additional_charges": {
      "value": null,
      "confidence": 0
    },
    "grand_total": {
      "value": null,
      "confidence": 0
    },
    "customer_name": {
      "value": null,
      "confidence": 0
    },
    "address": {
      "value": null,
      "confidence": 0
    }
  },
  "line_items": [],
  "validation_issues": [
    {
      "code": "MISSING_DOCUMENT_NUMBER",
      "message": "Required field 'document_number' is missing.",
      "severity": "ERROR",
      "field": "document_number"
    },
    {
      "code": "MISSING_DOCUMENT_DATE",
      "message": "Required field 'document_date' is missing.",
      "severity": "ERROR",
      "field": "document_date"
    },
    {
      "code": "MISSING_GRAND_TOTAL",
      "message": "Required field 'grand_total' is missing.",
      "severity": "ERROR",
      "field": "grand_total"
    },
    {
      "code": "LOW_CLASSIFICATION_CONFIDENCE",
      "message": "Document classification confidence is below the accepted threshold.",
      "severity": "WARNING",
      "field": "document_type"
    }
  ],
  "status": "MANUAL_REVIEW_REQUIRED"
}
```


Figure 10: API result for the degraded-document scenario

8.4 Scenario 4 – Deliberately Incorrect Grand Total

A copy of the clean invoice was modified so that the visible grand total no longer matched the subtotal and tax. The expected behavior is a GRAND_TOTAL_MISMATCH validation issue and MANUAL_REVIEW_REQUIRED status. The image itself is modified rather than editing the JSON output, so the complete OCR-to-validation pipeline is exercised.



INVOICE

East Repair Inc.
1912 Harvest Lane
New York, NY 12210

BILL TO
John Smith
2 Court Square
New York, NY 12210

SHIP TO
John Smith
3787 Pineview Drive
Cambridge, MA 12210

INVOICE # US-001
INVOICE DATE 11/02/2019
P.O.# 2312/2019
DUE DATE 26/02/2019

QTY	DESCRIPTION	UNIT PRICE	AMOUNT
1	Front and rear brake cables	100.00	100.00
2	New set of pedal arms	15.00	30.00
3	Labor 3hrs	5.00	15.00

Subtotal 145.00
Sales Tax 6.25% - - -
TOTAL $160.0

John Smith

TERMS & CONDITIONS
Payment is due within 15 days

Please make checks payable to: East Repair Inc.

Figure 11: Invoice with deliberately incorrect grand total


```
{
  "document_type": "INVOICE",
  "confidence": 0.878,
  "fields": {
    "document_number": {
      "value": "US-001",
      "confidence": 0.947
    },
    "document_date": {
      "value": "11/02/2019",
      "confidence": 0.641
    },
    "vendor_name": {
      "value": "East Repair Inc.",
      "confidence": 0.928
    },
    "currency": {
      "value": "$",
      "confidence": 0.893
    },
    "subtotal": {
      "value": 145,
      "confidence": 0.95
    },
    "discount": {
      "value": null,
      "confidence": 0
    },
    "tax": {
      "value": null,
      "confidence": 0
    },
    "additional_charges": {
      "value": null,
      "confidence": 0
    },
    "grand_total": {
      "value": 16,
      "confidence": 0.942
    },
    "customer_name": {
      "value": null,
      "confidence": 0
    },
    "address": {
      "value": null,
      "confidence": 0
    }
  },
```

```
  "line_items": [
    {
      "description": "Front and rear brake cables",
      "quantity": 1,
      "unit_price": 100,
      "line_total": 100,
      "confidence": 0.999
    },
    {
      "description": "New set of pedal arms",
      "quantity": 2,
      "unit_price": 15,
      "line_total": 30,
      "confidence": 0.994
    },
    {
      "description": "Labor 3hrs",
      "quantity": 3,
      "unit_price": 5,
      "line_total": 15,
      "confidence": 0.997
    }
  ],
  "validation_issues": [
    {
      "code": "GRAND_TOTAL_MISMATCH",
      "message": "Expected grand total 145.00 but found 16.",
      "severity": "ERROR",
      "field": "grand_total"
    }
  ],
  "status": "MANUAL_REVIEW_REQUIRED"
}
```


Figure 12: Validation output for the incorrect-total invoice

8.5 Scenario 5 – Missing Required Information

A test invoice was created with the document number removed while leaving the other information unchanged. The system is expected to produce MISSING_DOCUMENT_NUMBER and route the document to manual review. Similar fixtures were also prepared for missing date and missing total scenarios.



INVOICE

East Repair Inc.
1912 Harvest Lane
New York, NY 12210

BILL TO
John Smith
2 Court Square
New York, NY 12210

SHIP TO
John Smith
3787 Pineview Drive
Cambridge, MA 12210

INVOICE #	US-001
INVOICE DATE	11/02/2019
P.O.#	2312/2019
DUE DATE	26/02/2019

QTY	DESCRIPTION	UNIT PRICE	AMOUNT
1	Front and rear brake cables	100.00	100.00
2	New set of pedal arms	15.00	30.00
3	Labor 3hrs	5.00	15.00

Subtotal	145.00
Sales Tax 6.25%	- - -
TOTAL	

John Smith

TERMS & CONDITIONS
Payment is due within 15 days

Please make checks payable to: East Repair Inc.

Figure 13: Invoice with a missing required field

```
{
  "document_type": "INVOICE",
  "confidence": 0.857,
  "fields": {
    "document_number": {
      "value": "US-001",
      "confidence": 0.909
    },
    "document_date": {
      "value": "11/02/2019",
      "confidence": 0.641
    },
    "vendor_name": {
      "value": "East Repair Inc.",
      "confidence": 0.91
    },
    "currency": {
      "value": null,
      "confidence": 0
    },
    "subtotal": {
      "value": 145,
      "confidence": 0.95
    },
    "discount": {
      "value": null,
      "confidence": 0
    },
    "tax": {
      "value": null,
      "confidence": 0
    },
    "additional_charges": {
      "value": null,
      "confidence": 0
    },
    "grand_total": {
      "value": null,
      "confidence": 0
    },
    "customer_name": {
      "value": null,
      "confidence": 0
    },
    "address": {
      "value": null,
      "confidence": 0
    }
  },
  "line_items": [
    {
      "description": "Front and rear brake cables",
      "quantity": 1,
      "unit_price": 100,
      "line_total": 100,
      "confidence": 0.998
    },
    {
      "description": "New set of pedal arms",
      "quantity": 2,
      "unit_price": 15,
      "line_total": 30,
      "confidence": 0.997
    },
    {
      "description": "Labor 3hrs",
      "quantity": 3,
      "unit_price": 5,
      "line_total": 15,
      "confidence": 0.999
    }
  ],
  "validation_issues": [
    {
      "code": "MISSING_GRAND_TOTAL",
      "message": "Required field 'grand_total' is missing.",
      "severity": "ERROR",
      "field": "grand_total"
    }
  ],
  "status": "MANUAL_REVIEW_REQUIRED"
}
```


Figure 14: Validation output for the missing-field invoice

8.6 Scenario 6 – Unrelated Document

An unrelated restaurant/menu-style document was processed to test safe rejection behavior. The classifier returned UNKNOWN rather than forcing an invoice or quotation classification. The result was routed to MANUAL_REVIEW_REQUIRED because required invoice/quotation information was not available and classification confidence was not sufficient



Figure 15: Unrelated document used for UNKNOWN classification testing


```
{
  "document_type": "UNKNOWN",
  "confidence": 0.592,
  "fields": {
    "document_number": {
      "value": null,
      "confidence": 0
    },
    "document_date": {
      "value": null,
      "confidence": 0
    },
    "vendor_name": {
      "value": "A PLACE",
      "confidence": 0.799
    },
    "currency": {
      "value": null,
      "confidence": 0
    },
    "subtotal": {
      "value": null,
      "confidence": 0
    },
    "discount": {
      "value": null,
      "confidence": 0
    },
    "tax": {
      "value": null,
      "confidence": 0
    },
    "additional_charges": {
      "value": null,
      "confidence": 0
    },
    "grand_total": {
      "value": null,
      "confidence": 0
    },
    "customer_name": {
      "value": null,
      "confidence": 0
    },
    "address": {
      "value": null,
      "confidence": 0
    }
  },
```

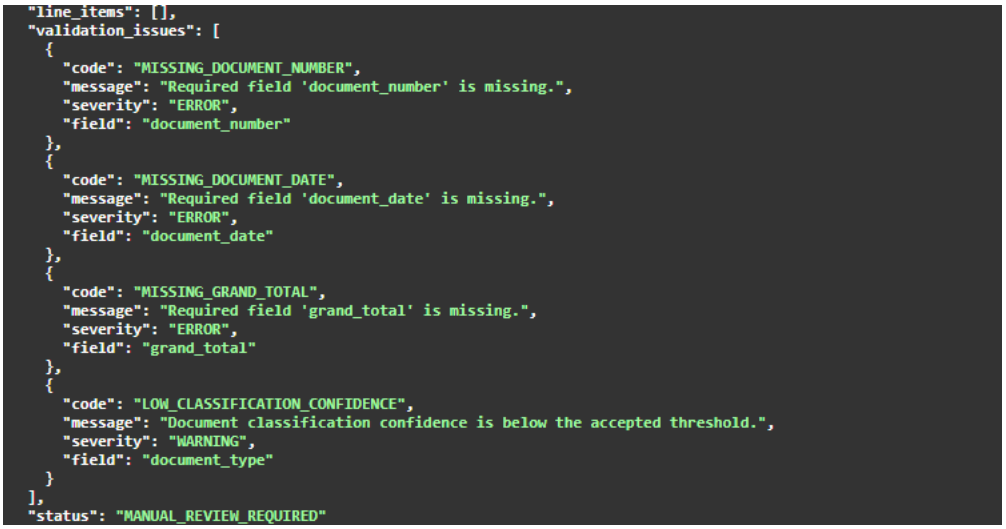


Figure 16: UNKNOWN / manual-review result for the unrelated document

8.7 Demonstration Results Summary

Scenario	Expected / Observed Result	Status
Valid invoice	Correct invoice classification, fields, tax, total, and line items	PROCESSED
Different quotation layout	Quotation classified and structured values extracted across a different table structure	PROCESSED
Rotated document	Preprocessing/OCR continues to handle the document	PROCESSED in tested rotated case
Low-resolution / blurred	Uncertainty is surfaced rather than silently accepted	MANUAL_REVIEW_REQUIRED
Incorrect total	GRAND_TOTAL_MISMATCH is generated	MANUAL_REVIEW_REQUIRED
Missing document number	MISSING_DOCUMENT_NUMBER is generated	MANUAL_REVIEW_REQUIRED
Unrelated document	Document remains UNKNOWN	MANUAL_REVIEW_REQUIRED

Table 3: Demonstration result summary

9 Dockerization

The application was packaged using a Dockerfile based on Python 3.11. The image installs the required system libraries, Python dependencies, and application files, then starts Uvicorn on port 8000. The Dockerized API was tested through /health, /docs, and a real invoice upload through /api/documents/analyze.

Build command:

```
docker build -t intelligent-document-engine
```

Run command:

```
docker run --rm -p 8000:8000 intelligent-document-engine
```

10 Safety and Reliability Considerations

- Missing values are not fabricated.
- Unknown documents are not forced into invoice or quotation categories.
- Financial mismatches are surfaced through validation issues.
- Unsupported/corrupt uploads are handled using controlled errors.
- No secret keys or credentials are required in the repository.
- Test/degraded fixtures are kept separate from production extraction logic.
- Field and document confidence information supports human review decisions.

11 Known Limitations

- OCR accuracy still depends on scan quality, font, resolution, and symbol recognition.
- Very complex or heavily stylized layouts may require additional extraction heuristics.
- Handwritten documents are not a primary target of the current prototype.
- Currency can remain null when OCR does not reliably recognize the symbol or explicit currency code.
- Duplicate-document detection is not implemented as durable cross-restart persistence in the current stateless prototype.
- The system is a document-understanding engine, not a complete accounting or ERP platform.

12 Future Improvements

- Add a persistent document registry or database-backed duplicate check where required by integration context.
- Add more diverse invoice and quotation layouts to the regression corpus.
- Improve date normalization while preserving the original OCR value and uncertainty.
- Add stronger table reconstruction for multi-line descriptions and complex merged cells.
- Add configurable confidence thresholds for different deployment environments.
- Pre-bundle OCR models in production images if fully offline startup is required.
- Add observability such as structured metrics, request tracing and performance measurements.

13 Conclusion

The Intelligent Invoice / Quotation Understanding Engine satisfies the core prototype objective by providing an end-to-end flow from document upload to structured, validated JSON output. The implementation combines preprocessing, OCR, evidence-based classification, spatial and semantic extraction, financial validation, confidence-aware review routing, FastAPI integration, automated testing and Docker packaging. The final robustness scenarios demonstrate that the system can process multiple layouts while safely flagging degraded, incomplete, inconsistent and unrelated documents for review.